Autonomous Interaction Management Framework: Comprehensi

Welcome to the technical guide and structural explanation of the **Autonomous Interaction Management Framework for Dynamic Site Handling** (Samsung Prime). This document serves as a complete file-by-file walkthrough of the entire project codebase.

The goal of this project is to build an autonomous web browser agent that can parse user intent, run a Chromium browser instance (via Playwright), extract interactive DOM elements, dynamically decide its next action using a Large Language Model (Groq/Llama-3), execute actions directly on the page, keep track of its history, and stop when the goal is achieved.

Codebase Architecture & Control Flow Overview

Before diving into individual files, it is helpful to visualize how control flows through the system.

Workflow Diagram

```
graph TD
    A[main.py - CLI Entry] -->|Parses user request| B[agents/reasoning_agent.py - ReasoningAgent]
    B -->|Calls parse| C[llm/intent_parser.py - IntentParser]
    C -->|Queries Groq API| D[llm/llm_client.py - LLMClient]
    B -->|Instructs launch & navigation| E[browser/controller.py - BrowserController]
    E -->|Opens Page| F[Playwright Browser Context]

    subgraph Autonomous Agent Loop [Max 15 Iterations]
        B -->|Extracts elements| G[browser/dom_extractor.py - DOMExtractor]
        G -->|Executes JavaScript on Page| F
        B -->|Formats text representation| H[context/context_builder.py - ContextBuilder]
        B -->|Retrieves past history| I[memory/history.py - MemoryState]
        B -->|Sends context & prompt| D
        D -->|Returns JSON response| B
        B -->|Validates Schema| J[models/action_models.py - AgentAction]
        B -->|Executes action| K[browser/actions.py - BrowserExecutor]
        K -->|Interacts with page| F
        B -->|Captures progress| F
    end
```

Flow of Control Step-by-Step:

1. **User Request**: The program is executed via `main.py` where a user prompt (e.g., *"extract the price of a light in the attic"*) is input.
1. **Intent Parsing**: `ReasoningAgent` delegates the raw text prompt to `IntentParser`, which structures the intent (determining the target URL, the core search term, etc.) using `LLMClient`.
1. **Browser Setup**: `BrowserController` initializes a headless or headful Chromium instance via Playwright and navigates to the starting URL.
1. **Autonomous Execution Loop**:
    - **DOM Extraction**: `DOMExtractor` executes a specialized in-page JavaScript script to search for visible, interactive elements (buttons, inputs, links), assigns them temporary `data-playwright-id` attributes, and returns their details.
    - **Context Building**: `ContextBuilder` transforms these elements into a lightweight text block.
    - **Memory Integration**: `MemoryState` adds previous steps and extracted data to prevent loops and provide

continuity.
    - **LLM Choice**: The LLM analyzes the current page layout and selects the next logical action (e.g., click an element, type into a search bar, scroll, navigate, or finalize with `done`).
    - **Action Execution**: `BrowserExecutor` translates this decision into Playwright commands (e.g., clicking on a locator with `[data-playwright-id='pw-id-X']`).
    - **Progress Capture**: A screenshot is captured for debugging, and memory is updated.
1. **Termination**: The agent exits when the LLM issues a `done` action or the maximum iteration limit is reached.

Detailed File-by-File Breakdown

1. Root Files

[main.py](file:///d:/Summer%20Kartik/Samsung%20prime/main.py)

- **Purpose & Role**:
  This is the main entry point of the CLI application. It handles user input collection (command-line arguments or interactive terminal prompts), initializes logging, constructs the `ReasoningAgent`, and initiates the asynchronous execution flow.
- **Control Flow**:
    - Starts by setting up the console logging output and mapping logs to `execution_history.log`.
    - Reads command-line arguments. If empty, runs an interactive terminal CLI prompting the user.
    - Instantiates `ReasoningAgent(headless=False, max_iterations=15)`.
    - Triggers `agent.execute_task(query)` and runs it inside Python's async event loop.
- **Code Explanation**:
    - `main()`: A coroutine that acts as the starting hub.
    - `logger.add("execution_history.log", rotation="10 MB")`: Directs system logs to a local file, ensuring it rotates when it grows too large.
    - `agent = ReasoningAgent(headless=False, max_iterations=15)`: Constructs the runner, configured to launch a visible browser (`headless=False`) so the developer can watch actions happen.
- **Pending Tasks & Improvements**:
    - [ ] **Configurable CLI Parameters**: Add flags (e.g., `--headless`, `--max-iterations`, `--model`) using `argparse` instead of basic string slicing of `sys.argv`.
    - [ ] **Interactive REPL**: Add a step-by-step review mode where the user must approve/reject each agent action before execution.

[requirements.txt](file:///d:/Summer%20Kartik/Samsung%20prime/requirements.txt)

- **Purpose & Role**:
  Lists all Python packages and libraries required by the project.
- **Control Flow**:
    - Used by `pip` or package managers to install dependencies.
- **Code Explanation**:
    - `playwright`: Dynamic browser control.
    - `beautifulsoup4` & `lxml`: HTML parsing.
    - `fastapi` & `uvicorn`: Web API serving (unused).
    - `openai`: Client wrapper for API connections.
    - `loguru`: Preconfigured, descriptive logging.
    - `chromadb`: Vector Database client (unused).

    - `pydantic`: Schema serialization and data validation.
    - `python-dotenv`: Management of system environment variables.
- **Pending Tasks & Improvements**:
    - [ ] **Clean Unused Dependencies**: Remove unused libraries like `fastapi`, `uvicorn`, and `chromadb` if they are not going to be implemented, or construct their implementations.
    - [ ] **Pin Versions**: Pin exact package versions to avoid breaking changes in downstream libraries (e.g., Playwright and OpenAI clients).

[.env](file:///d:/Summer%20Kartik/Samsung%20prime/.env)

- **Purpose & Role**:
  Stores sensitive credential configuration variables locally (API keys, models) to prevent them from leaking into the source control.
- **Control Flow**:
    - Read at startup by `python-dotenv` during `LLMClient` initialization.
- **Code Explanation**:
    - `GROQ_API_KEY`: Groq API authorization key.
    - `MODEL_NAME`: Name of the LLM model to query.
- **Pending Tasks & Improvements**:
    - [ ] **Security Vulnerability**: The actual API key `gsk_...` is currently hardcoded and checked in. The key must be immediately revoked and replaced with a placeholder (e.g. `GROQ_API_KEY=your_key_here`).
    - [ ] **Validation**: Add a verification step at startup to check if `GROQ_API_KEY` is present and valid before attempting LLM runs.

[.gitignore](file:///d:/Summer%20Kartik/Samsung%20prime/.gitignore)

- **Purpose & Role**:
  Tells Git which files and folders to ignore in the repository.
- **Control Flow**:
    - Read by git client.
- **Code Explanation**:
    - Standard Python system files (`__pycache__`, venv folders).
    - Web execution logs (`execution_history.log`, `*.log`).
    - Playwright browser binary caches (`.local-browsers/`).
    - Local credentials (`.env`).
    - Runtime runtime artifacts (`screenshot.png`, `final_state_*.png`).
- **Pending Tasks & Improvements**:
    - None. It is complete.

---

2. Agent Component

[agents/reasoning_agent.py](file:///d:/Summer%20Kartik/Samsung%20prime/agents/reasoning_agent.py)

- **Purpose & Role**:
  The core conductor of the autonomous framework. It binds the browser controller, the DOM extractor, the context generator, the memory manager, the LLM caller, and the action executor together into a stateful, iterative loop.
- **Control Flow**:
    - `execute_task(user_query)` starts by calling `IntentParser`.
    - Navigates the browser page to the extracted starting site URL.

    - Enters a loop running up to `max_iterations` times.
    - In each iteration:
        1. Extracts visible elements via `DOMExtractor`.
        1. Builds a context prompt using `ContextBuilder` and current page URLs.
        1. Merges memory states (previous actions, data).
        1. Formulates a system and user prompt and feeds it to `LLMClient`.
        1. Deserializes the LLM's raw response using the `AgentAction` model.
        1. Calls `BrowserExecutor.execute(action)` to execute the step.
        1. Captures a debugging screenshot (`screenshot.png`).
        1. Exits early if `action.action == "done"` or on major browser failures.
- **Code Explanation**:
    - `default_cli_printer(event_type, data)`: A utility callback that writes structured events (actions, extracted details) to the logs in a human-readable format.
    - `ReasoningAgent.__init__(...)`: Hooks up the sub-components and configures maximum iteration cycles.
    - `_emit(...)` / `_emit_log(...)` / `_emit_screenshot()`: Utilities to propagate events (useful if wrapped inside a graphical user interface) and save screenshot logs.
    - `execute_task(user_query)`: Executes the full cycle loop.
- **Pending Tasks & Improvements**:
    - [ ] **Generic Fallback Search**: Replace the hardcoded target fallback (`"books.toscrape.com"`) with a search query execution on a search engine (like DuckDuckGo/Google) if no target website is resolved.
    - [ ] **Better Token Optimization**: The DOM elements can exceed token budgets for long webpages. A mechanism is needed to summarize elements or rank them before feeding them to the prompt.
    - [ ] **Smart Loop Recovery**: If the agent gets stuck in a loop of identical actions, the agent should notice and self-correct (currently, it just keeps repeating until it hits max iterations).

---

3. Browser Component

[browser/controller.py](file:///d:/Summer%20Kartik/Samsung%20prime/browser/controller.py)

- **Purpose & Role**:
  Handles low-level browser operations (launching, creating profiles, contexts, loading pages, waiting for network idle, and closing the instance).
- **Control Flow**:
    - Instantiated by `ReasoningAgent`.
    - Spawns Chromium when `open_website(url)` is invoked.
    - Keeps a reference to the active `Page` and `BrowserContext` for use by the DOM extractor and executor.
- **Code Explanation**:
    - `launch_browser()`: Starts an async Playwright instance, launches Chromium, establishes a standardized desktop user agent and viewport size (1280x800) to ensure uniform element layout.
    - `open_website(url)`: Loads the target page, waiting until the main frame's `load` event is triggered.
    - `close_browser()`: Safely closes contexts, pages, and browser processes to prevent dangling headless processes in memory.
- **Pending Tasks & Improvements**:
    - [ ] **Multi-Browser Support**: Add option to switch between WebKit (Safari), Firefox, and Chromium based on runtime configurations.
    - [ ] **Persistent Browser Context**: Add support for loading user data directories (cookies, storage) to persist logged-in sessions across executions.
    - [ ] **Network State Waiting**: Implement options to wait for `networkidle` state instead of just standard DOM

loading, ensuring slow, heavy JavaScript websites render fully before actions start.

[browser/dom_extractor.py](file:///d:/Summer%20Kartik/Samsung%20prime/browser/dom_extractor.py)

- **Purpose & Role**:
  Scans the active browser page to identify elements that the LLM agent can interact with. It filters out irrelevant DOM noise to save token space.
- **Control Flow**:
  - Called inside the reasoning agent's loop.
  - Executes a single, optimized JavaScript script on the browser page.
  - Returns a list of structured node dictionaries representing the interactive nodes.
- **Code Explanation**:
  - `isVisible(el)`: Helper inside JS that filters out elements whose CSS display is `none`, visibility is `hidden`, opacity is `0`, or which have physical sizes of 0 width or 0 height.
  - `isChildOfInteractive(el)`: Checks if an element resides inside an active `<a>` or `<button>` block, skipping duplicates to keep the context clean (unless the child node is a nested input/textarea).
  - `isInsideFooter(el)`: Skips non-input/non-button elements located within footer zones to focus LLM focus on primary content regions.
  - `el.setAttribute('data-playwright-id', uniqueId)`: **Crucial Step**. Inserts a custom attribute `data-playwright-id` directly on the webpage DOM. The value is standard (`pw-id-0`, `pw-id-1`, etc.). This acts as a highly reliable bridge between the LLM client (which selects elements by index) and Playwright selectors.
- **Pending Tasks & Improvements**:
  - [ ] **Web Components / Shadow DOM Support**: The current CSS selector queries do not penetrate Shadow Roots, meaning widgets built with modern frameworks like Lit or custom Web Components will appear blank to the agent.
  - [ ] **Modals & Overlays Prioritization**: If an overlay/modal is open, the extractor still extracts background elements. The extractor needs a way to filter out elements covered by a backdrop.
  - [ ] **Dynamic Truncation**: Truncating text content to a static 100 characters can cut off important semantic details. A smarter summarization system would prevent losing key data.

[browser/actions.py](file:///d:/Summer%20Kartik/Samsung%20prime/browser/actions.py)

- **Purpose & Role**:
  The action parser and executor. It receives the pydantic `AgentAction` resolved by the LLM and translates it into physical interaction events on the Playwright `Page` object.
- **Control Flow**:
  - `execute(action)` inspects `action.action`.
  - Handles page-level operations (`wait`, `navigate`, `back`, `done`).
  - Resolves element selectors using the `data-playwright-id` attribute or standard text fallbacks.
  - Executes targeted commands on the locator (such as `click()`, `fill()`, `scroll_into_view_if_needed()`, or `text_content()`).
- **Code Explanation**:
  - `execute(action)`: Inspects action names and branches to appropriate Playwright async APIs.
  - `selector = f"[data-playwright-id='{action.target}']"`: Selects the targeted element by its custom ID.
  - `self.page.locator(selector).first`: Obtains the first matching locator handle.
  - `await locator.wait_for(state="attached", timeout=5000)`: Ensures the element is rendered and ready before clicking or typing to avoid race conditions.
  - `text = await locator.text_content(...)`: Executes text scrapers during `extract` actions.
- **Pending Tasks & Improvements**:
  - [ ] **Complex Actions**: Add actions for `hover` (mouse hovering), `drag_to` (drag and drop), `press_key` (e.g. pressing Enter inside forms), and select option menus.

- [ ] **Select Element / Dropdowns**: Add explicit capability to manipulate `<select>` dropdown tags.
- [ ] **Multi-match Recovery**: Currently, it defaults to `.first`. If a target points to multiple overlapping items, it should throw an alert or resolve them cleanly.

---

4. Context & LLM Components

[context/context_builder.py](file:///d:/Summer%20Kartik/Samsung%20prime/context/context_builder.py)

- **Purpose & Role**:
  Structures the DOM elements into a clean, text-only outline optimized for LLM reading.
- **Control Flow**:
  - Takes raw node dictionaries from `DOMExtractor` and stringifies them.
- **Code Explanation**:
  - `build_context(current_url, elements)`: Constructs a document starting with the current URL followed by a serialized list of the interactive elements.
  - Condenses key element properties (`tag`, `text`, `placeholder`, `aria_label`, `type`) into a single spaced line per element.
- **Pending Tasks & Improvements**:
  - [ ] **Hierarchical Layout Context**: Presenting elements as a flat list loses information about visual grouping. We should format the context with structural markers (e.g., nesting or markdown indentation) to show parent-child hierarchies.
  - [ ] **Token Trimming**: Implement pagination or thresholding if the context string length exceeds context window limits.

[llm/llm_client.py](file:///d:/Summer%20Kartik/Samsung%20prime/llm/llm_client.py)

- **Purpose & Role**:
  Communicates with the Groq API endpoint using the OpenAI SDK client interface.
- **Control Flow**:
  - Reads configuration from `.env`.
  - Connects to Groq at `https://api.groq.com/openai/v1`.
  - Sends raw prompts and returns parsed JSON responses.
- **Code Explanation**:
  - `AsyncOpenAI(api_key=self.api_key, base_url="https://api.groq.com/openai/v1")`: Configures OpenAI's client to target Groq's high-speed completion service.
  - `response_format={"type": "json_object"}`: Forces the LLM to output valid JSON text structure.
  - `temperature=0.0`: Locks model generation temperature to zero to ensure deterministic, logical web actions.
- **Pending Tasks & Improvements**:
  - [ ] **Model Provider Router**: Implement support for other model providers (such as Gemini, Anthropic, or OpenAI) via runtime environment toggles.
  - [ ] **Automatic Retry Handling**: Groq has strict rate-limits. Integrate retry loops with exponential backoff on HTTP status `429`.

[llm/intent_parser.py](file:///d:/Summer%20Kartik/Samsung%20prime/llm/intent_parser.py)

- **Purpose & Role**:
  Acts as the pre-processor. Parses the user's natural language goal to determine the starting URL and query focus.
- **Control Flow**:

    - Receives the raw query string from `ReasoningAgent`.
    - Queries `LLMClient` using a system parsing prompt.
    - Returns a structured `ParsedIntent` instance.
- **Code Explanation**:
    - `ParsedIntent(BaseModel)`: Pydantic model with fields like `website` (name), `website_url` (address), `intent` (action type), and `query` (search keys).
    - `IntentParser.parse(user_query)`: Generates a JSON mapping intent fields and constructs a `ParsedIntent` model.
- **Pending Tasks & Improvements**:
    - [ ] **Semantic URL Mapping**: Include a local dictionary mapping common website names (e.g., Google, Amazon, Github) to their URLs to avoid querying the LLM for simple lookups.

---

5. Memory & Data Components

[memory/history.py](file:///d:/Summer%20Kartik/Samsung%20prime/memory/history.py)

- **Purpose & Role**:
  Keeps track of visited locations, past actions, and extracted data to provide stateful context to the agent.
- **Control Flow**:
    - Accessed by `ReasoningAgent` during context building.
- **Code Explanation**:
    - `MemoryState`: Houses `actions_history`, `visited_urls`, and `extracted_data`.
    - `get_context_string()`: Returns the last 5 executed actions as a text list for the LLM prompt.
- **Pending Tasks & Improvements**:
    - [ ] **Vector Search Database**: Integrate the `chromadb` client (currently listed in `requirements.txt` but unused) to index page contents and support semantic lookup of long-term history.
    - [ ] **State Persistence**: Save memory states to disk (`.json`) so a task run can be paused, analyzed, or resumed later.

[memory/__init__.py](file:///d:/Summer%20Kartik/Samsung%20prime/memory/__init__.py)

- **Purpose & Role**: Exposes classes to simplify importing from the package root.
- **Code Explanation**: Exports `MemoryState`.
- **Pending Tasks & Improvements**: None.

---

6. Action Models

[models/action_models.py](file:///d:/Summer%20Kartik/Samsung%20prime/models/action_models.py)

- **Purpose & Role**:
  Defines the schema for actions issued by the LLM.
- **Code Explanation**:
    - `AgentAction(BaseModel)`: Fields include:
        - `action`: Restricts choice to `click`, `type`, `scroll`, `wait`, `navigate`, `extract`, `back`, or `done`.
        - `target`: The target element (e.g. `pw-id-X`).
        - `value`: String payload.
        - `reasoning`: The explanation behind the chosen action.
- **Pending Tasks & Improvements**:

- [ ] **Dynamic Action Extension**: Allow adding custom actions dynamically without altering the core model schema.

[models/__init__.py](file:///d:/Summer%20Kartik/Samsung%20prime/models/__init__.py)

- **Purpose & Role**: Exports model schemas for easier import.
- **Code Explanation**: Exports `AgentAction`.
- **Pending Tasks & Improvements**: None.

---

7. Core Directories (Unused/Structure)

[utils/ (Empty)](file:///d:/Summer%20Kartik/Samsung%20prime/utils)

- **Purpose**: Intended for general utility modules.
- **Pending Tasks**:
  - [ ] Add helpers like HTML text sanitizers, visual coordinate calculators, or logger configurations.

[verification/ (Empty)](file:///d:/Summer%20Kartik/Samsung%20prime/verification)

- **Purpose**: Intended for running test suites and verifying agent tasks.
- **Pending Tasks**:
  - [ ] Add visual comparison assertions, unit tests for DOM filtering logic, or regression checks.

Complete Project Flow Visualization

Below is the visual map showing how control and data flow between all components during an execution cycle:

```
sequenceDiagram
    autonumber
    actor User as User CLI
    participant M as main.py
    participant A as ReasoningAgent (agents/reasoning_agent.py)
    participant IP as IntentParser (llm/intent_parser.py)
    participant LC as LLMClient (llm/llm_client.py)
    participant BC as BrowserController (browser/controller.py)
    participant DE as DOMExtractor (browser/dom_extractor.py)
    participant CB as ContextBuilder (context/context_builder.py)
    participant MS as MemoryState (memory/history.py)
    participant BE as BrowserExecutor (browser/actions.py)

    User->>M: Executes program (query)
    M->>A: Instantiates ReasoningAgent & runs execute_task(query)
    A->>IP: parse(query)
    IP->>LC: generate_json(system_prompt, user_query)
    LC-->>IP: return ParsedIntent JSON
    IP-->>A: return ParsedIntent object
    A->>BC: open_website(website_url)
    BC-->>A: Browser is Launched & Navigated

    loop Autonomous Decision Loop (Max 15 iterations)
        A->>DE: extract_interactive_elements()
```

```
        DE-->>A: return List of Visible Elements (with pw-id injected)
        A->>CB: build_context(current_url, elements)
        CB-->>A: return Cleaned String Representation
        A->>MS: get_context_string()
        MS-->>A: return Memory History Context String
        A->>LC: generate_json(agent_system_prompt, user_prompt)
        LC-->>A: return Decided Action JSON
        A->>BE: execute(action)
        BE-->>A: execute action on webpage
        A->>MS: add_action(action)
        Note over A: Capture screenshot.png
    end

    A->>BC: close_browser()
    A-->>M: Completed Task
    M-->>User: Exit program
```